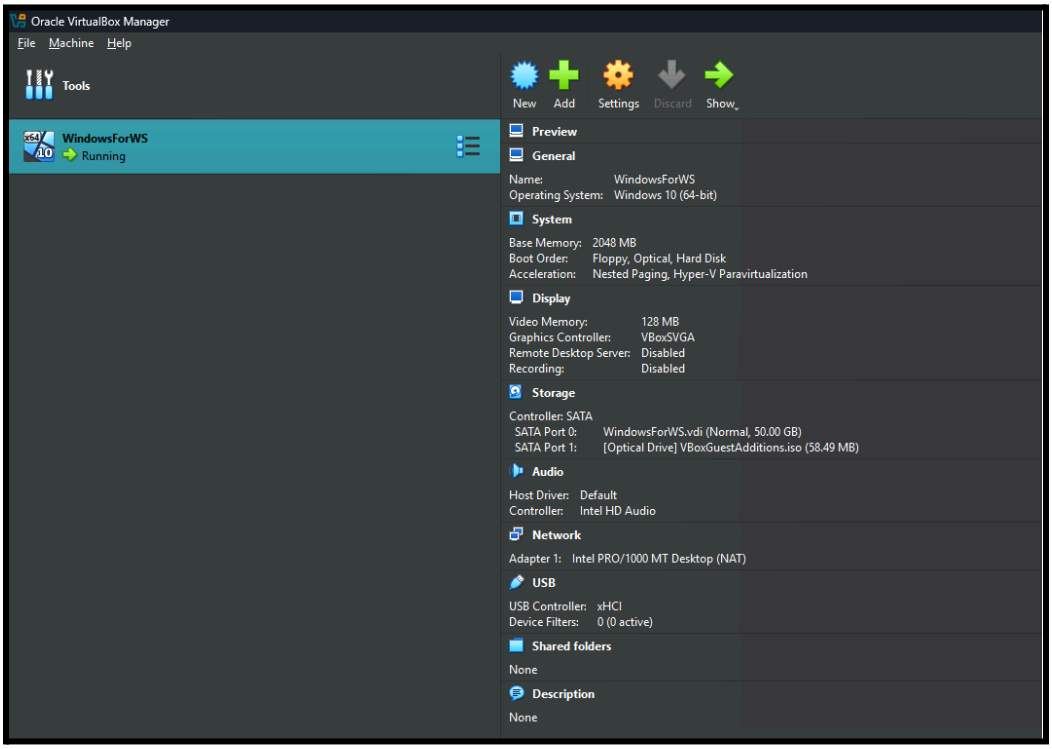
Name: Mansi Ramchandra Mourya
Roll no: 31011123031
Class: TY Bsc Computer Science

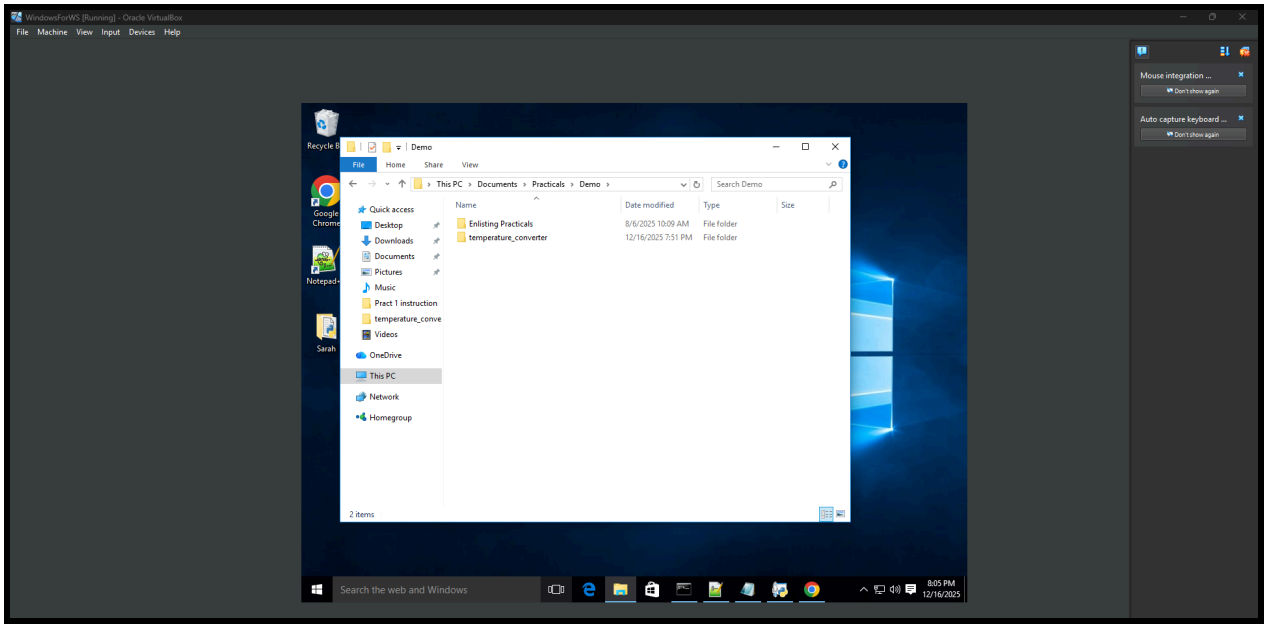
https://tinyurl.com/tycs10001

WEB SERVICES
Practical 1

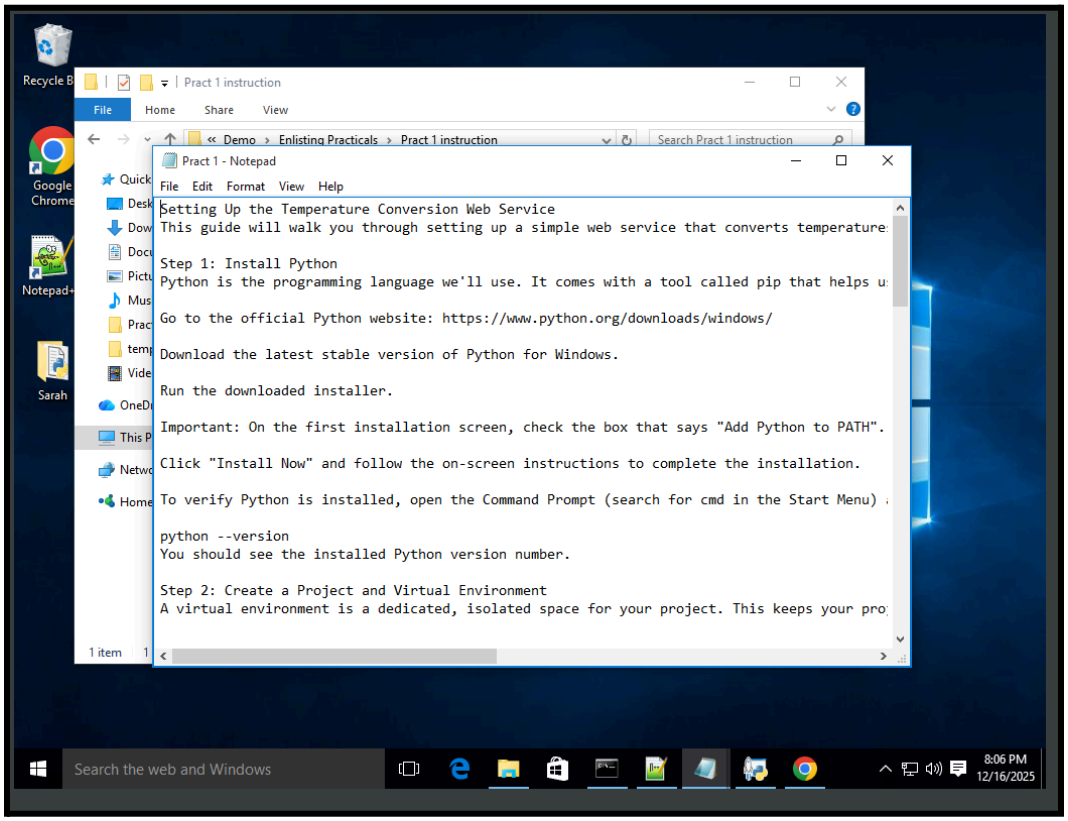
Temperature Converter:-

Open Oracle Virtual Box:

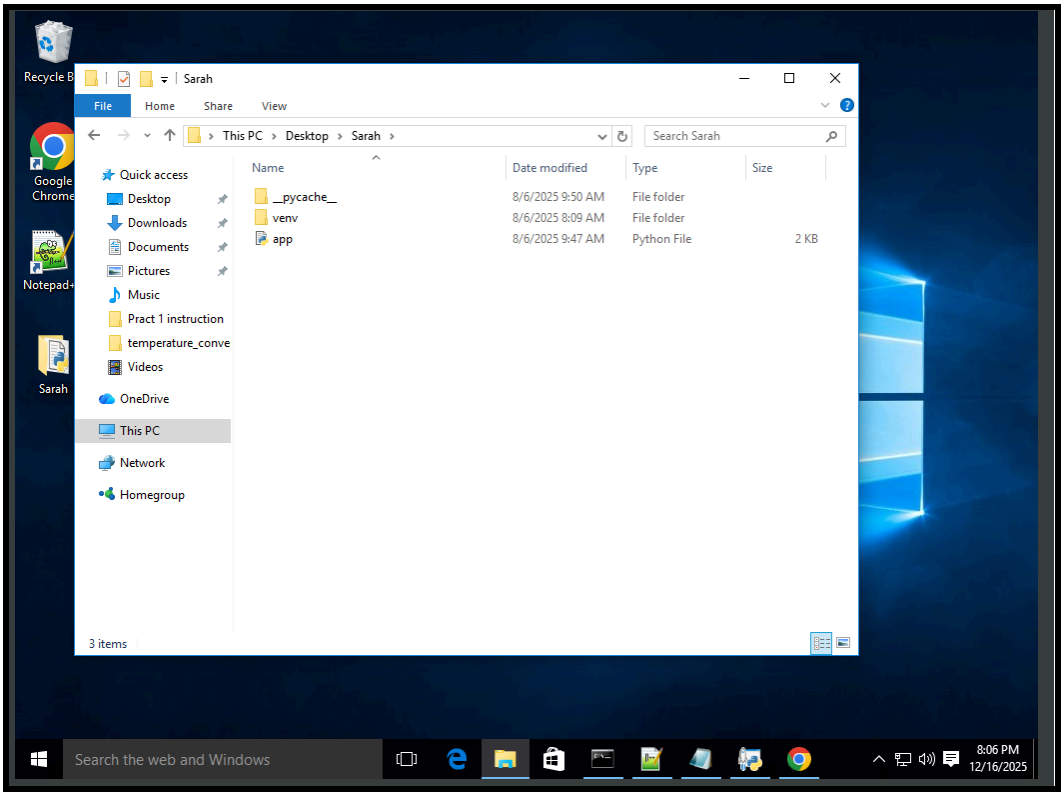




Instructions in VM:

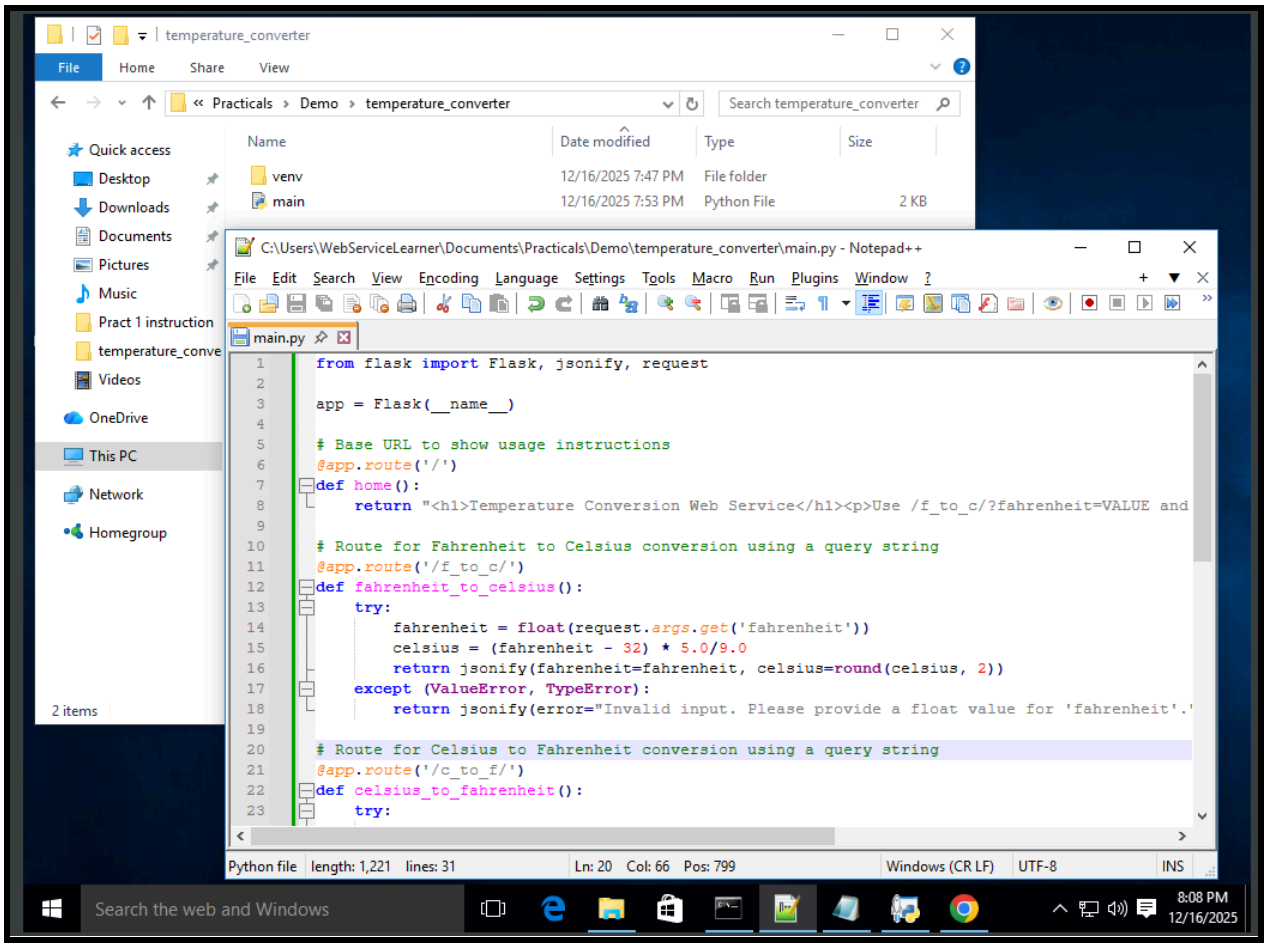


ctrl+X in different folder

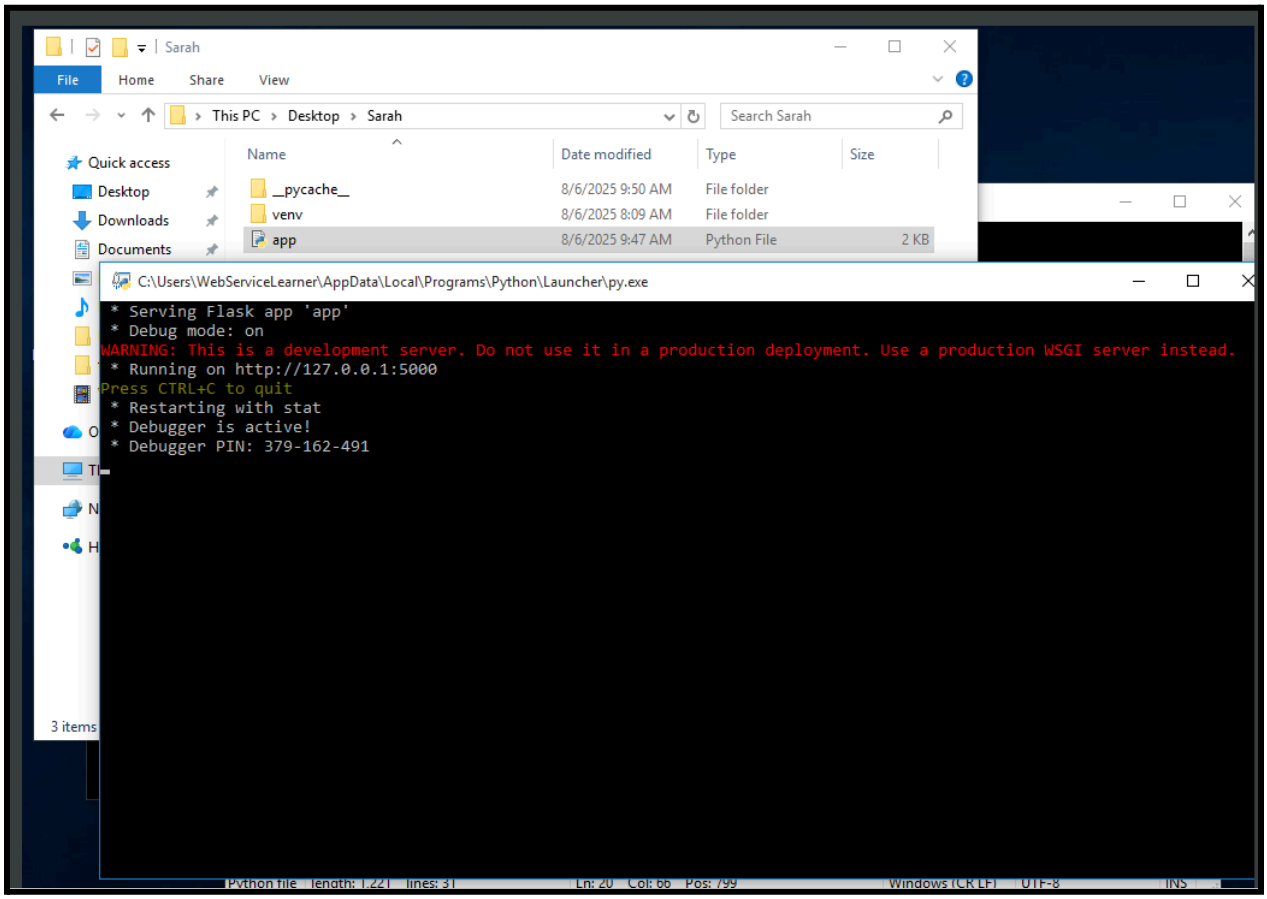


- python -m venv venv
- pip install Flask

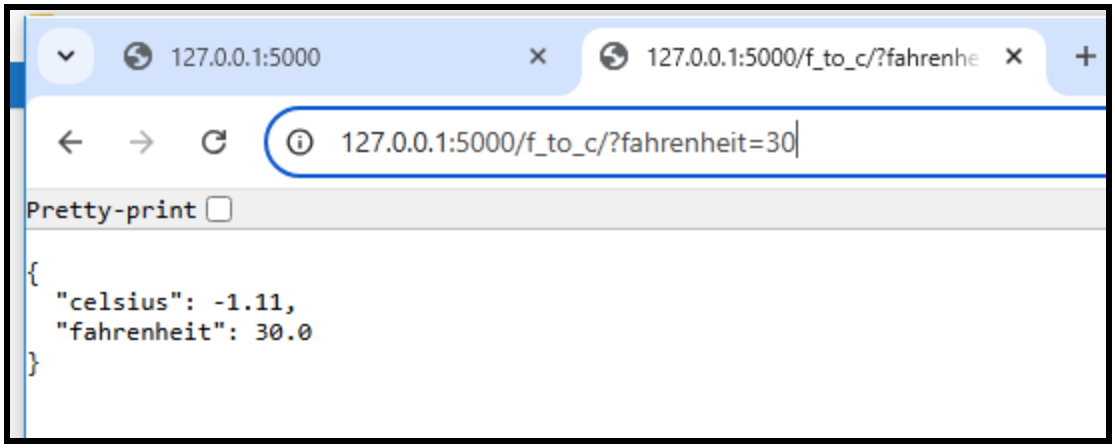
Put the code in a Python file and save it


temperature_converter
File Home Share View
Practicals › Demo › temperature_converter
Search temperature_converter
Quick access
Desktop
Downloads
Documents
Pictures
Music
Pract 1 instruction
temperature_conve
Videos
OneDrive
This PC
Network
Homegroup
2 items
Name Date modified Type Size
venv 12/16/2025 7:47 PM File folder
main 12/16/2025 7:53 PM Python File 2 KB
C:\Users\WebServiceLearner\Documents\Practicals\Demo\temperature_converter\main.py - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
main.py
from flask import Flask, jsonify, request
app = Flask(__name__)
# Base URL to show usage instructions
@app.route('/')
def home():
return "<h1>Temperature Conversion Web Service</h1><p>Use /f_to_c/?fahrenheit=VALUE and
Route for Fahrenheit to Celsius conversion using a query string
@app.route('/f_to_c/')
def fahrenheit_to_celsius():
try:
fahrenheit = float(request.args.get('fahrenheit'))
celsius = (fahrenheit - 32) * 5.0/9.0
return jsonify(fahrenheit=fahrenheit, celsius=round(celsius, 2))
except (ValueError, TypeError):
return jsonify(error="Invalid input. Please provide a float value for 'fahrenheit'.'
Route for Celsius to Fahrenheit conversion using a query string
@app.route('/c_to_f/')
def celsius_to_fahrenheit():
try:
Python file length: 1,221 lines: 31 Ln: 20 Col: 66 Pos: 799 Windows (CR LF) UTF-8 INS
Search the web and Windows
8:08 PM
12/16/2025

Then open the .py file and you'll get this…copy the http and paste it in a browser.



Then put a forward slash in the url and add this
/f_to_c/?fahrenheit=VALUE



Your output is here!

Temperature converter code:-

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Base URL to show usage instructions
@app.route('/')
def home():
    return "<h1>Temperature Conversion Web Service</h1><p>Use /f_to_c/?fahrenheit=VALUE and /c_to_f/?celsius=VALUE to convert temperatures.</p>"

# Route for Fahrenheit to Celsius conversion using a query string
@app.route('/f_to_c/')
def fahrenheit_to_celsius():
    try:
        fahrenheit = float(request.args.get('fahrenheit'))
        celsius = (fahrenheit - 32) * 5.0/9.0
        return jsonify(fahrenheit=fahrenheit, celsius=round(celsius, 2))
    except (ValueError, TypeError):
```

```
        return jsonify(error="Invalid input. Please provide a float value for
'fahrenheit'.")

# Route for Celsius to Fahrenheit conversion using a query string
@app.route('/c_to_f/')
def celsius_to_fahrenheit():
    try:
        celsius = float(request.args.get('celsius'))
        fahrenheit = (celsius * 9.0/5.0) + 32
        return jsonify(celsius=celsius, fahrenheit=round(fahrenheit, 2))
    except (ValueError, TypeError):
        return jsonify(error="Invalid input. Please provide a float value for
'celsius'.")

if __name__ == '__main__':
    app.run(debug=True)
```

Practical 1.1

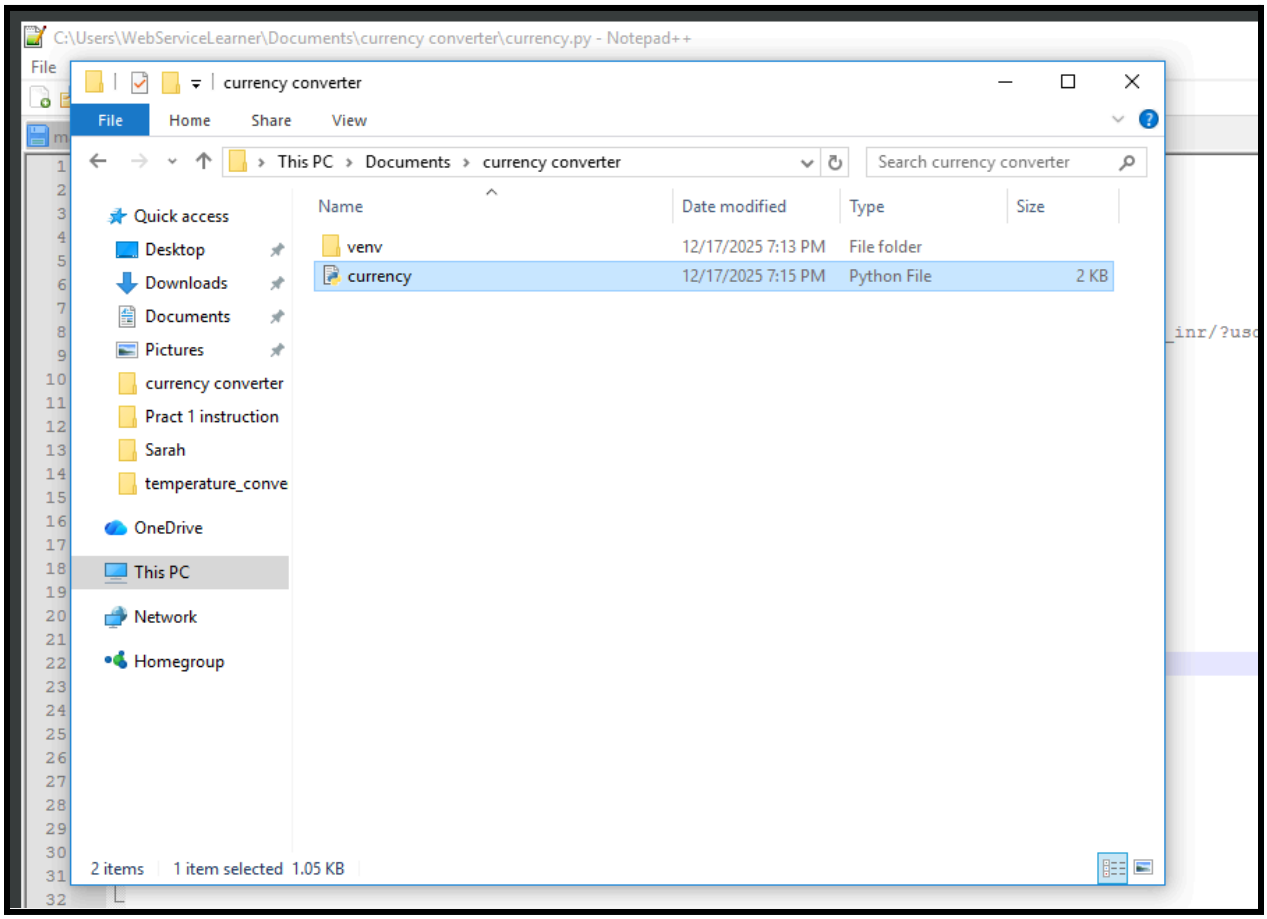
Currency Converter:-

currency.py code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)
# Base URL to show usage instructions
@app.route('/')
def home():
    return "<h1>Currency Conversion Web Service</h1><p>Use
/inr_to_usd/?inr=VALUE and /usd_to_inr/?usd=VALUE to convert
currencies.</p>"
# Route for Fahrenheit to Celsius conversion using a query string

@app.route('/inr_to_usd/')
def inr_to_usd():
    try:
        inr = float(request.args.get('inr'))
        usd = inr / 83
        return jsonify(inr=inr, usd=round(usd, 2))
    except (ValueError, TypeError):
        return jsonify(error="Invalid input. Please provide a float value for
'inr'.")

# Route for Celsius to Fahrenheit conversion using a query string
@app.route('/usd_to_inr/')
def usd_to_inr():
    try:
        usd = float(request.args.get('usd'))
        inr= usd *84
        return jsonify(usd=usd, inr=round(inr, 2))
    except (ValueError, TypeError):
```

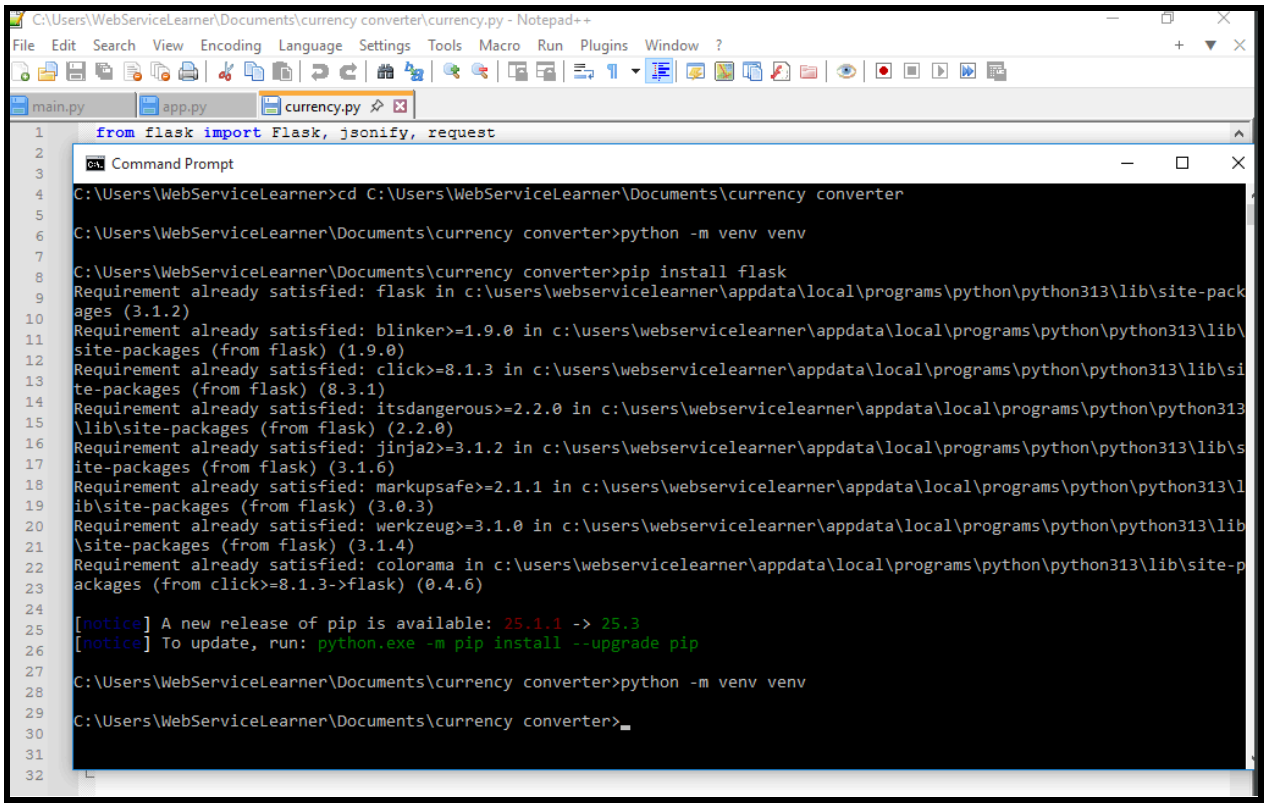
```
        return jsonify(error="Invalid input. Please provide a float value for
'usd'.")

if __name__ == '__main__':
    app.run(debug=True)
```

Create a new Folder..

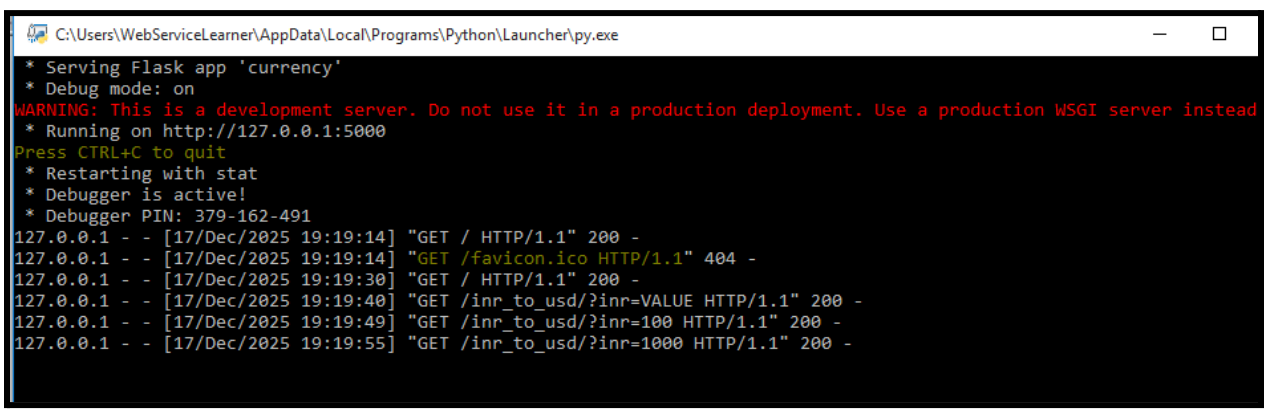


Open cmd and type..

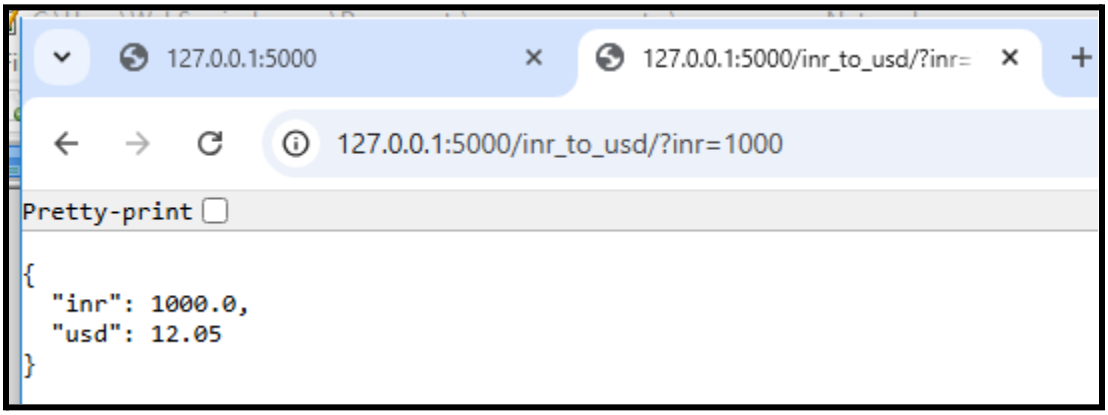
- python -m venv venv
- pip install Flask



Open the currency.py file



127.0.0.1:5000
127.0.0.1:5000/inr_to_usd/?inr=
127.0.0.1:5000
Currency Conversion Web Service
Use /inr_to_usd/?inr=VALUE and /usd_to_inr/?usd=VALUE to convert currencies.


127.0.0.1:5000
127.0.0.1:5000/inr_to_usd/?inr=
127.0.0.1:5000/inr_to_usd/?inr=1000
Pretty-print
{
"inr": 1000.0,
"usd": 12.05
}

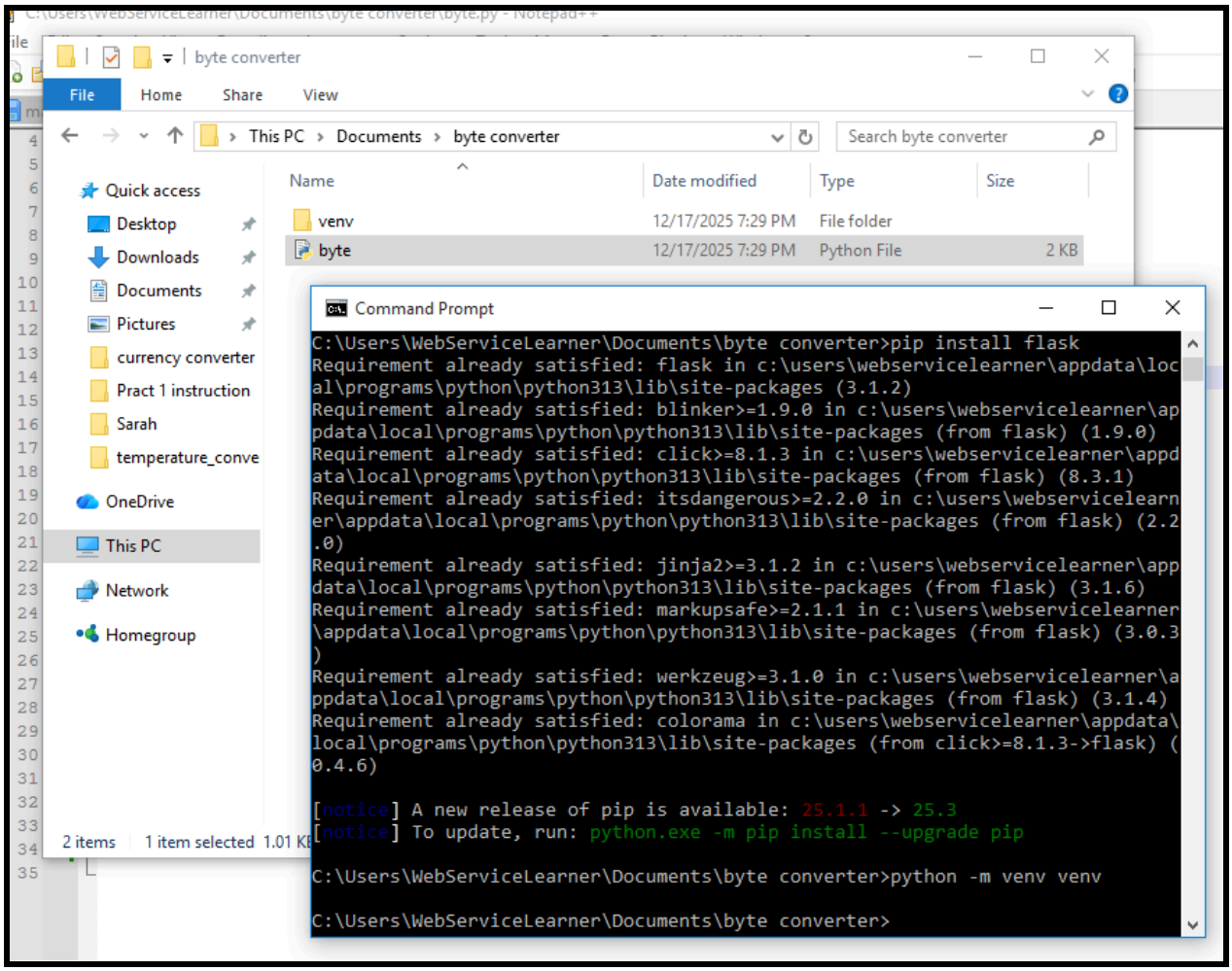
Practical 1.2

Terabyte-Gigabyte Converter:-

byte.py code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Base URL to show usage instructions
@app.route('/')
def home():
    return (
        "<h1>Data Size Conversion Web Service</h1>"
        "<p>Use /tb_to_gb/?tb=VALUE and /gb_to_tb/?gb=VALUE to convert
data sizes.</p>"
    )

# Route for Terabyte to Gigabyte conversion
@app.route('/tb_to_gb/')
def tb_to_gb():
    try:
        tb = float(request.args.get('tb'))
        gb = tb * 1024
        return jsonify(tb=tb, gb=round(gb, 2))
    except (ValueError, TypeError):
        return jsonify(error="Invalid input. Please provide a float value for 'tb'.")

# Route for Gigabyte to Terabyte conversion
@app.route('/gb_to_tb/')
def gb_to_tb():
    try:
        gb = float(request.args.get('gb'))
        tb = gb / 1024
```

```
        return jsonify(gb=gb, tb=round(tb, 4))
    except (ValueError, TypeError):
        return jsonify(error="Invalid input. Please provide a float value for
'gb'.")

if __name__ == '__main__':
    app.run(debug=True)
```



Open cmd and type..

- python -m venv venv
- pip install Flask

```
C:\Users\WebServiceLearner\AppData\Local\Programs\Python\Launcher\py.exe
 * Serving Flask app 'byte'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server ins
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 379-162-491
127.0.0.1 - - [17/Dec/2025 19:29:58] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/Dec/2025 19:30:28] "GET /tb_to_gb/?tb=8 HTTP/1.1" 200 -
```

127.0.0.1:5000 | 127.0.0.1:5000/inr_to | 127.0.0.1:5000 | 127.0.0.1:5000//tb_to

127.0.0.1:5000

Data Size Conversion Web Service

Use /tb_to_gb/?tb=VALUE and /gb_to_tb/?gb=VALUE to convert data sizes.

127.0.0.1:5000 | 127.0.0.1:5000/inr_to | 127.0.0.1:5000 | 127.0.0.1:5000//tb_to

127.0.0.1:5000//tb_to_gb/?tb=8

Pretty-print


```
{
  "gb": 8192.0,
  "tb": 8.0
}
```

Practical 1.3

Distance Calculator:-

Map api
Loc A lat long
Loc B lat long
VV to churchgate

```
from flask import Flask, jsonify, request

import requests


app = Flask(__name__)


DISTANCE_MATRIX_URL = "https://api.distancematrix.ai/maps/api/distancematrix/json"

API_KEY = "8Kq6ul7Cp2FKkhzIMKWwpfVdFFddPf8uQs4eQK4fK9C588CO6fhzRDeZpHL6eJaB"


@app.route('/get_distance', methods=['GET'])

def get_distance():

    origin = request.args.get('origin')

    destination = request.args.get('destination')


    if not origin or not destination:

        return jsonify({"error": "Please provide both origin and destination as query parameters"}), 400



    params = {

        'origins': origin,

        'destinations': destination,
```

```
        'key': API_KEY
    }
        response = requests.get(DISTANCE_MATRIX_URL, params=params)

    if response.status_code == 200:
        data = response.json()
        return jsonify(data)
    else:
        return jsonify({"error": "Failed to get distance data", "status_code": response.status_code}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

Output :


```
{
  "destination_addresses": [
    "Parking lot, IIT Area, Powai, Mumbai, Maharashtra 400076"
  ],
  "origin_addresses": [
    "K J Somaiya Institute of Dharma Studies, Somaiya Campus, Vidyanagar, Vidya Vihar East, Vidyavihar, Mumbai, Maharashtra 400077,
India"
  ],
  "rows": [
    {
      "elements": [
        {
          "destination": "19.133253069369125, 72.91505245059443",
          "distance": {
            "text": "13.7 km",
            "value": 13677
          },
          "duration": {
            "text": "29 mins",
            "value": 1793
          },
          "origin": "19.07282695940927, 72.89780042361116",
          "status": "OK"
        }
      ]
    }
  ],
  "status": "OK"
}
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return jsonify(message="Web Services Lab is running")

if __name__ == "__main__":
    app.run(debug=True)

@app.route("/c_to_f")
def c_to_f():
    c = float(request.args.get("celsius"))
    f = (c * 9/5) + 32
    return jsonify(celsius=c, fahrenheit=round(f, 2))


@app.route("/f_to_c")
def f_to_c():
    f = float(request.args.get("fahrenheit"))
    c = (f - 32) * 5/9
    return jsonify(fahrenheit=f, celsius=round(c, 2))

@app.route("/bmi")
def bmi():
    weight = float(request.args.get("weight"))
    height = float(request.args.get("height"))

    bmi_value = weight / (height ** 2)

    return jsonify(
        weight_kg=weight,
        height_m=height,
        bmi=round(bmi_value, 2)
    )

@app.route("/mile_to_km")
def mile_to_km():
    miles = float(request.args.get("miles"))
    km = miles * 1.60934
    return jsonify(miles=miles, kilometers=round(km, 2))
```

```
@app.route("/km_to_mile")
def km_to_mile():
    km = float(request.args.get("km"))
    miles = km / 1.60934
    return jsonify(kilometers=km, miles=round(miles, 2))


@app.route("/gb_to_tb")
def gb_to_tb():
    gb = float(request.args.get("gb"))
    tb = gb / 1024
    return jsonify(gb=gb, tb=round(tb, 4))


@app.route("/tb_to_gb")
def tb_to_gb():
    tb = float(request.args.get("tb"))
    gb = tb * 1024
    return jsonify(tb=tb, gb=round(gb, 2))

@app.route("/inr_to_usd")
def inr_to_usd():
    inr = float(request.args.get("inr"))
    usd = inr * 0.012

    return jsonify(
        rupees=inr,
        dollars=round(usd, 2)
    )
```

Run the code in cmd using:

flask --app app.py --debug run

Name : Mansi Ramchandra Mourya
Seat No. : 31011123031

Web Service
Practical 2

Setting Up an Arithmetic Web Service

This guide will show you how to build a basic web service using Python and Flask that can perform addition, subtraction, multiplication, and division.

Step 1: Install Python

Python is the programming language that powers our web service. It comes with pip, a tool for installing other necessary libraries.

Download Python: Go to the official Python website:
https://www.python.org/downloads/windows/

Run the Installer: Download and run the latest stable version of Python for Windows.

+++

Add to PATH: On the very first screen of the installer, make sure to check the box that says "Add Python to PATH". This is crucial as it allows you to run Python and pip commands directly from your Command Prompt.

Complete Installation: Click "Install Now" and follow any remaining prompts.

Verify Installation: Open the Command Prompt (search for cmd in the Start Menu) and type:

```
python --version
```

You should see the installed Python version number, confirming it's set up correctly.

Step 2: Create a Project and Virtual Environment

A virtual environment creates an isolated space for your project. This prevents conflicts between the libraries your current project needs and those used by other Python projects on your computer.

Create Project Folder: Make a new folder for your project. For example, you can create one in your Documents folder and name it arithmetic_service.

Navigate to Folder: Open the Command Prompt and use the cd command to go into your new project folder. For example:

```
cd C:\Users\YourUser\Documents\arithmetic_service
```

(Replace YourUser with your actual Windows username).

Create Virtual Environment: Inside your project folder, create the virtual environment:

```
python -m venv venv
```

This command creates a new subfolder named venv containing the isolated Python environment.

Activate Virtual Environment: To start using this isolated environment, run:

```
venv\Scripts\activate
```

You'll know it's active when you see (venv) at the beginning of your command prompt line, like (venv) C:\Users\YourUser\Documents\arithmetic_service>.

Step 3: Install the Flask Library

Flask is a lightweight web framework for Python that makes it easy to create web applications and services. We need to install it specifically within our active virtual environment.

With your virtual environment active (you should see (venv) in your command prompt), run the following command:

pip install Flask

Pip will download and install Flask and any other libraries it needs into your venv folder.

Step 4: Create the Web Service Code

Now, we'll write the Python code that defines our arithmetic web service. This code will handle incoming requests and perform calculations.

Open Text Editor: Open a text editor (like Notepad, VS Code, or Sublime Text).

Create app.py: Create a new file and save it as app.py directly inside your arithmetic_service project folder.

Paste Code: Copy and paste the following Python code exactly into your app.py file.This code defines routes for addition, subtraction, multiplication, and division, using query string parameters for the numbers.

1. Create a RESTful Web Service using Flask in Replit that calculates arithmetic operations.

CODE-

```
from flask import Flask, jsonify, request

app = Flask(__name__)
```

```
# Base URL to show usage instructions

@app.route('/')

def home():

    return "<h1>Arithmetic Web Service</h1><p>Use the following
endpoints with query parameters
(?num1=VALUE&num2=VALUE):</p><ul><li>/add</li><li>/subtract</li><li>/
multiply</li><li>/divide</li></ul><p>Example: /add?num1=10&num2=5</p>"

def parse_numbers():

    """Helper function to parse num1 and num2 from request arguments."""

    try:

        num1 = float(request.args.get('num1'))

        num2 = float(request.args.get('num2'))

        return num1, num2

    except (ValueError, TypeError):

        return None, None # Indicate invalid input

@app.route('/add')

def add():

    num1, num2 = parse_numbers()

    if num1 is None or num2 is None:

        return jsonify(error="Invalid input. Please provide valid float values for
'num1' and 'num2'."), 400

    result = num1 + num2

    return jsonify(operation="add", num1=num1, num2=num2, result=result)

@app.route('/subtract')

def subtract():

    num1, num2 = parse_numbers()
```

```
    if num1 is None or num2 is None:
        return jsonify(error="Invalid input. Please provide valid float values for 'num1' and 'num2'."), 400
    result = num1 - num2
    return jsonify(operation="subtract", num1=num1, num2=num2, result=result)
@app.route('/multiply')
def multiply():
    num1, num2 = parse_numbers()
    if num1 is None or num2 is None:
        return jsonify(error="Invalid input. Please provide valid float values for 'num1' and 'num2'."), 400
    result = num1 * num2
    return jsonify(operation="multiply", num1=num1, num2=num2, result=result)
@app.route('/divide')
def divide():
    num1, num2 = parse_numbers()
    if num1 is None or num2 is None:
        return jsonify(error="Invalid input. Please provide valid float values for 'num1' and 'num2'."), 400
    if num2 == 0:
        return jsonify(error="Cannot divide by zero."), 400
    result = num1 / num2
    return jsonify(operation="divide", num1=num1, num2=num2, result=result)
if __name__ == '__main__':
```

app.run(debug=True)

OUTPUT-

```
C:\Users\WebServiceLearner>cd C:\Users\WebServiceLearner\Desktop

C:\Users\WebServiceLearner\Desktop>python -m venv venv

C:\Users\WebServiceLearner\Desktop>venv\Scripts\activate
```

```
(venv) C:\Users\WebServiceLearner\Desktop>pip install Flask
Collecting Flask
```

```
(venv) C:\Users\WebServiceLearner\Desktop\first>flask --app app.py --debug run
 * Serving Flask app 'app.py'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 880-925-447
127.0.0.1 - - [07/Jan/2026 19:21:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Jan/2026 19:21:21] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [07/Jan/2026 19:21:50] "GET /add?num1=10&num2=5 HTTP/1.1" 200 -
127.0.0.1 - - [07/Jan/2026 19:22:06] "GET /subtract?num1=10&num2=5 HTTP/1.1" 200 -
```

Arithmetic Web Service

Use the following endpoints with query parameters (?num1=VALUE&num2=VALUE):

- /add
- /subtract
- /multiply
- /divide

Example: /add?num1=10&num2=5

```
{
  "num1": 10.0,
  "num2": 5.0,
  "operation": "add",
  "result": 15.0
}
```

```
{
  "num1": 10.0,
  "num2": 5.0,
  "operation": "subtract",
  "result": 5.0
}
```

```
{
  "num1": 10.0,
  "num2": 5.0,
  "operation": "multiply",
  "result": 50.0
}
```

```
{
  "num1": 10.0,
  "num2": 5.0,
  "operation": "divide",
  "result": 2.0
}
```

Name: Mansi Ramchandra Mourya

Roll no: 31011123031

Class: TY Bsc Computer Science

https://tinyurl.com/tycs10001

WEB SERVICES

Practical 3

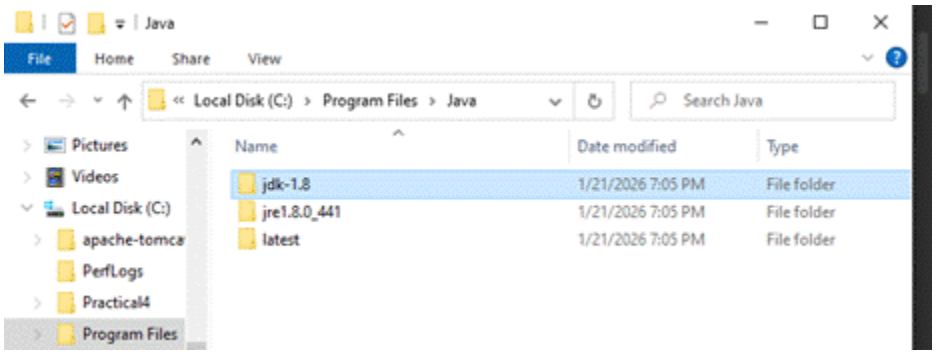
Step 1: Install & Configure JDK 8

1. Install JDK

Run the downloaded installer.

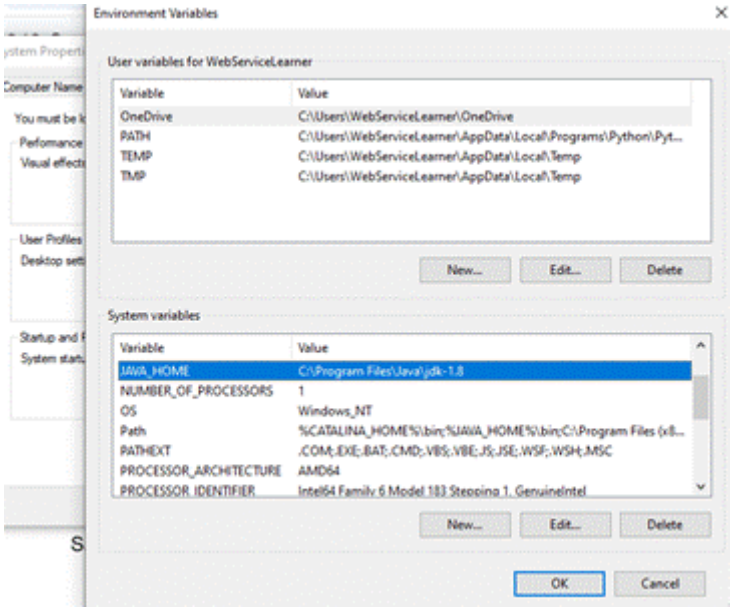
Keep clicking Next → Next → Finish.

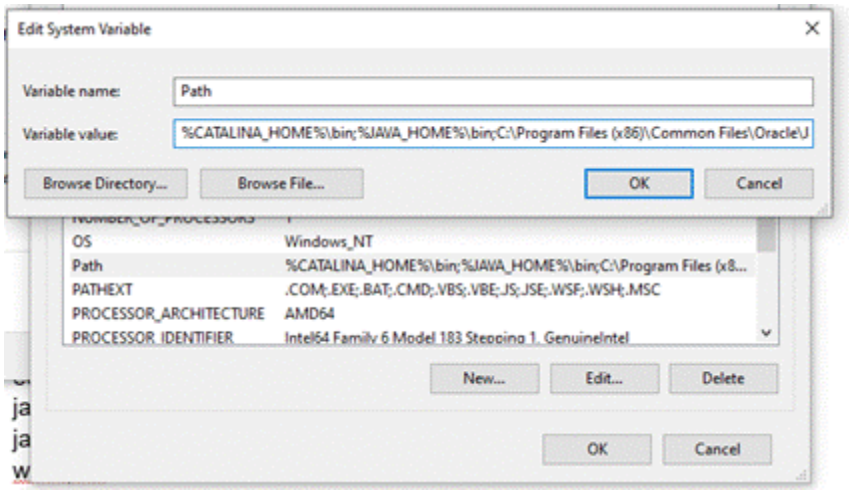
Default install path:



2. Set Environment Variables (baby steps)

- Copy the full path (e.g., C:\Program Files\Java\jdk1.8.0_441)
- Under System Variables → click New....
- Variable name: JAVA_HOME
- Variable value: C:\Program Files\Java\jdk1.8.0_441
- Still in System Variables → find Path → click Edit.
- %JAVA_HOME%\bin;
- Remove old/duplicate Java paths (like C:\Program Files (x86)\CommonFiles\Oracle\Java\java8path).





Open new Command Prompt (important!) and check:

echo %JAVA_HOME%

java -version

javac -version

wsimport -version

Expected output:

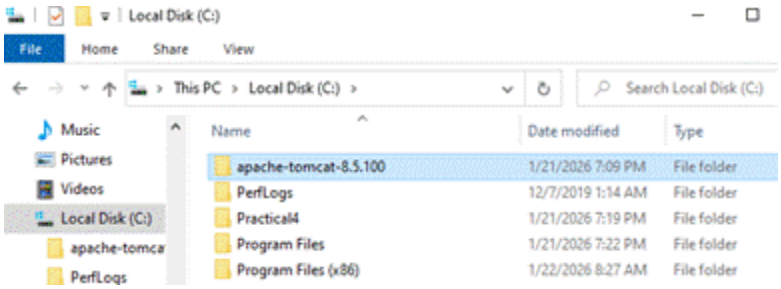
```
C:\Program Files\Java\jdk1.8.0_441
java version "1.8.0_441"
javac 1.8.0_441
JAX-WS RI 2.2.x
```

Step 2: Install & Configure Tomcat (Optional for Servlet Version):

Apache Tomcat 8.5.100 (zip file):

http://archive.apache.org/dist/tomcat/tomcat-8/v8.5.100/bin/apache-tomcat-8.5.100-windows-x64.zip

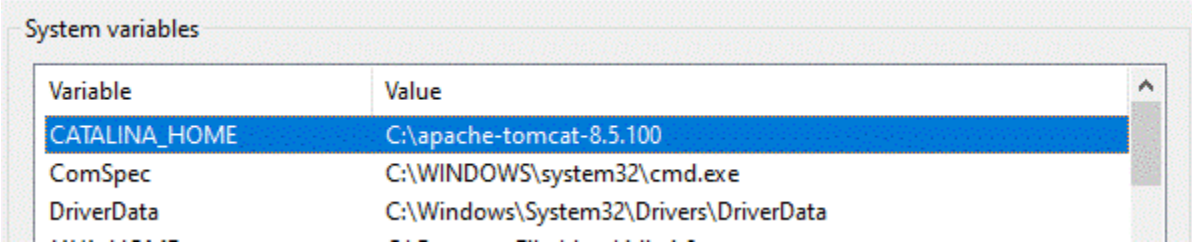
Extract the Tomcat zip to:



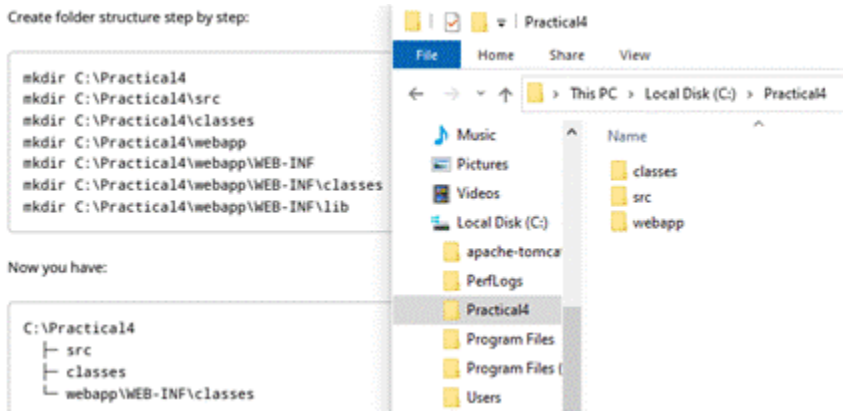
- In Environment Variables → System Variables → New…
- Name: CATALINA_HOME
- Value: C:\apache-tomcat

- Edit Path → add:

%CATALINA_HOME%\bin



Step 3: Prepare Project Folder:



Step 4: Generate SOAP Client Stubs:

Run in CMD:

- cd C:\Practical4\src
- wsimport -keep -p calculator http://www.dneonline.com/calculator.asmx?WSDL

If no internet, save WSDL locally and run:

wsimport -keep -p calculator file:///C:/Practical4/src/calculator.wsdl

This generates files in:

C:\Practical4\src\calculator

Step 5: Create Console Client Program:

File: C:\Practical4\src\CalculatorClient.java

```
import calculator.Calculator;
import calculator.CalculatorSoap;
public class CalculatorClient {
public static void main(String[] args) {
Calculator service = new Calculator();
CalculatorSoap port = service.getCalculatorSoap();
int sum = port.add(10, 20);
int mul = port.multiply(5, 6);
System.out.println("10 + 20 = " + sum);
System.out.println("5 * 6 = " + mul);
```

}

}

Step 6: Compile & Run Console Client:

Compile:

- cd C:\Practical4\src
- javac -d ..\classes CalculatorClient.java calculator*.java

Run:

```
C:\Practical4\src>javac -d ..\classes CalculatorClient.java calculator\*.java

C:\Practical4\src>cd ..\classes

C:\Practical4\classes>java CalculatorClient
10 + 20 = 30
5 * 6 = 30
```

Step 7: (Optional) Servlet Version with Tomcat:

1. Add index.jsp

File: C:\Practical4\webapp\index.jsp

```
<html><body>

<h2>SOAP Calculator</h2>
```

```
<form action="calc" method="get">
A: <input name="a" value="10"/>  
B: <input name="b" value="15"/>  
<input type="submit" value="Calculate"/>
</form>
</body></html>
```

2. Add web.xml

File: C:\Practical4\webapp\WEB-INF\web.xml

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
version="3.1">
<servlet>
<servlet-name>CalcServlet</servlet-name>
<servlet-class>yourapp.CalculatorServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>CalcServlet</servlet-name>
<url-pattern>/calc</url-pattern>
</servlet-mapping>
```

```
<welcome-file-list>
<welcome-file>index.jsp</welcome-file>
</welcome-file-list>
</web-app>
```

3. Create Servlet

File: C:\Practical4\src\yourapp\CalculatorServlet.java

```
package yourapp;
import calculator.Calculator;
import calculator.CalculatorSoap;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.*;
public class CalculatorServlet extends HttpServlet {
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)

throws ServletException, IOException {
resp.setContentType("text/html;charset=UTF-8");
```

```
try {
int a = Integer.parseInt(req.getParameter("a"));
int b = Integer.parseInt(req.getParameter("b"));
Calculator service = new Calculator();
CalculatorSoap port = service.getCalculatorSoap();
int result = port.add(a, b);

resp.getWriter().println("<html><body>");
resp.getWriter().printf("%d + %d = %d", a, b, result);
resp.getWriter().println("</body></html>");
} catch (Exception e) {
e.printStackTrace(resp.getWriter());
}
}
}
```

4. Compile and place classes:

- cd C:\Practical4
- javac -d webapp\WEB-INF\classes src\calculator*.java
- javac -cp "%CATALINA_HOME%\lib\servlet-api.jar;webapp\WEB-INF\classes" -d webapp\WEB-INF\classes src\yourapp\CalculatorServlet.java

```
C:\Practical4>javac -cp "%CATALINA_HOME%\lib\servlet-api.jar;webapp\WEB-INF\classes" -d weba
pp\WEB-INF\classes src\yourapp\CalculatorServlet.java
```

5. Package WAR and deploy:

- cd C:\Practical4\webapp
- jar -cvf calcapp.war *

copy calcapp.war %CATALINA_HOME%\webapps

```
C:\Practical4>cd C:\Practical4\webapp

C:\Practical4\webapp>jar -cvf calcapp.war *
added manifest
adding: index.jsp(in = 220) (out= 150)(deflated 31%)
adding: WEB-INF/(in = 0) (out= 0)(stored 0%)
adding: WEB-INF/classes/(in = 0) (out= 0)(stored 0%)
adding: WEB-INF/classes/calculator/(in = 0) (out= 0)(stored 0%)
adding: WEB-INF/classes/calculator/Add.class(in = 773) (out= 431)(deflated 44%)
adding: WEB-INF/classes/calculator/AddResponse.class(in = 743) (out= 419)(deflated 43%)
adding: WEB-INF/classes/calculator/Calculator.class(in = 2250) (out= 1031)(deflated 54%)
adding: WEB-INF/classes/calculator/CalculatorSoap.class(in = 1588) (out= 626)(deflated 60%)
adding: WEB-INF/classes/calculator/Divide.class(in = 782) (out= 437)(deflated 44%)
adding: WEB-INF/classes/calculator/DivideResponse.class(in = 764) (out= 419)(deflated 45%)
adding: WEB-INF/classes/calculator/Multiply.class(in = 788) (out= 438)(deflated 44%)
adding: WEB-INF/classes/calculator/MultiplyResponse.class(in = 778) (out= 424)(deflated 45%)
adding: WEB-INF/classes/calculator/ObjectFactory.class(in = 1303) (out= 531)(deflated 59%)
adding: WEB-INF/classes/calculator/package-info.class(in = 315) (out= 234)(deflated 25%)
adding: WEB-INF/classes/calculator/Subtract.class(in = 788) (out= 437)(deflated 44%)
adding: WEB-INF/classes/calculator/SubtractResponse.class(in = 778) (out= 424)(deflated 45%)
adding: WEB-INF/classes/yourapp/(in = 0) (out= 0)(stored 0%)
adding: WEB-INF/classes/yourapp/CalculatorServlet.class(in = 1516) (out= 813)(deflated 46%)
adding: WEB-INF/lib/(in = 0) (out= 0)(stored 0%)
adding: WEB-INF/web.xml(in = 564) (out= 241)(deflated 57%)

C:\Practical4\webapp>copy calcapp.war %CATALINA_HOME%\webapps
        1 file(s) copied.
```

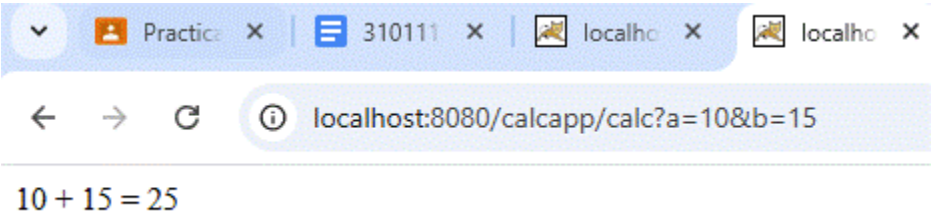
6. Start Tomcat and test:

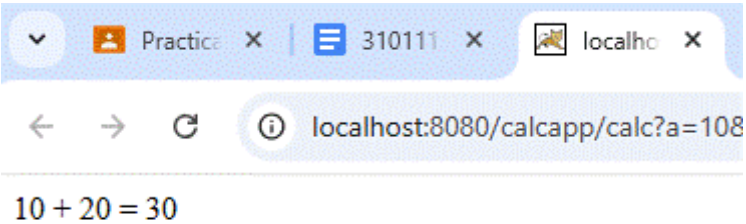
- cd %CATALINA_HOME%\bin
- startup.bat

```
C:\Practical4\webapp>cd %CATALINA_HOME%\bin

C:\apache-tomcat-8.5.100\bin>startup.bat
Using CATALINA_BASE:   "C:\apache-tomcat-8.5.100"
Using CATALINA_HOME:   "C:\apache-tomcat-8.5.100"
Using CATALINA_TMPDIR: "C:\apache-tomcat-8.5.100\temp"
Using JRE_HOME:        "C:\Program Files\Java\jdk-1.8"
Using CLASSPATH:       "C:\apache-tomcat-8.5.100\bin\bootstrap.jar;C:\apache-tomcat-8.5.100\bin\tomcat-juli.jar"
Using CATALINA_OPTS:   ""
C:\apache-tomcat-8.5.100\bin>
```

Browser: -

http://localhost:8080/calcapp/
http://localhost:8080/calcapp/calc?a=10&b=20





Name: Mansi Ramchandra Mourya
Roll no: 31011123031
Class: TY Bsc Computer Science

WEB SERVICES
Practical 4

Title

Develop a Desktop Application Client Who Consumes Web Services

Objective

To create a small Java desktop (Swing) application that consumes both a SOAP web service and a REST web service. The application demonstrates interoperability across different platforms.

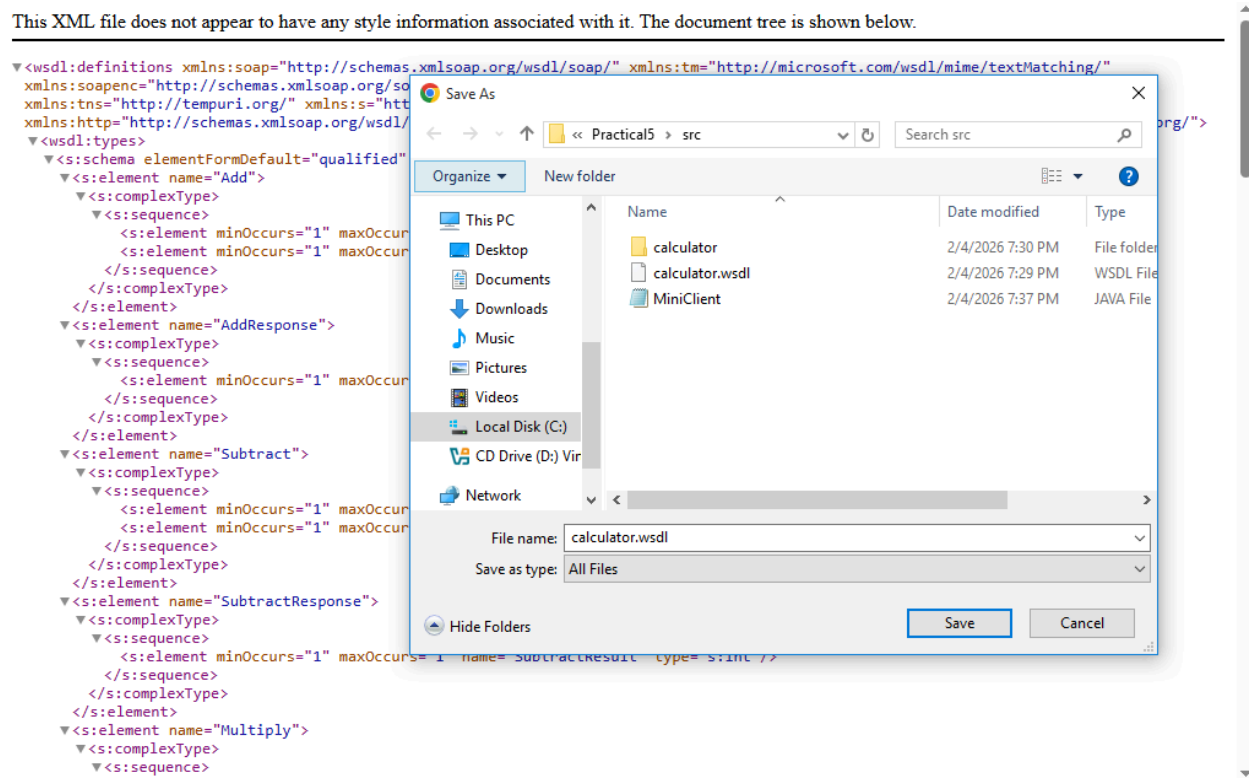
Step-by-Step Instructions

Step 1 — Create folders

```
C:\Windows\system32>mkdir C:\Practical5

C:\Windows\system32>cd C:\Practical5

C:\Practical5>mkdir src

C:\Practical5>mkdir classes

C:\Practical5>cd src
```

Step 2 — Save WSDL file locally

1. Open in browser: http://www.dneonline.com/calculator.asmx?WSDL
2. Save as calculator.wsdl inside: C:\Practical5\src



Step 3 — Generate SOAP stubs

cd C:\Practical4\src
wsimport -keep -p calculator file:///C:/Practical5/src/calculator.wsdl

```
C:\Practical5>cd src

C:\Practical5\src>wsimport -keep -p calculator file:///C:/Practical5/src/calculator.wsdl
parsing WSDL...


[WARNING] Ignoring SOAP port "CalculatorSoap12": it uses non-standard SOAP 1.2 binding.
You must specify the "-extension" option to use this binding.
  line 192 of file:/C:/Practical5/src/calculator.wsdl


Generating code...


Compiling code...
```

Step 4 — Create Java application

Create C:\Practical4\src\MiniClient.java and paste the following code:

```
import javax.swing.*;
import java.awt.*;
import java.net.*;
```

```
import calculator.Calculator;
import calculator.CalculatorSoap;

public class MiniClient {
   static String http(String u) throws Exception {
      java.util.Scanner s = new java.util.Scanner(new URL(u).openStream(), "UTF-8");
      String r = s.useDelimiter("\\A").next(); s.close(); return r;
   }
   static String dogUrl() throws Exception {
      String j = http("https://dog.ceo/api/breeds/image/random");
      return j.split("\"message\":\"")[1].split("\"")[0].replace("\\/", "/");
   }
   public static void main(String[] a) throws Exception {
      Calculator c = new Calculator();
      int sum = c.getCalculatorSoap().add(5,7);

      JFrame f = new JFrame("Practical 5 — SOAP 5+7 = " + sum);
      JLabel img = new JLabel("Click the button to load a dog ", SwingConstants.CENTER);
      JButton b = new JButton("Fetch Dog");
      f.add(img, BorderLayout.CENTER); f.add(b, BorderLayout.SOUTH);
      f.setSize(500,400); f.setLocationRelativeTo(null);
      f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); f.setVisible(true);

      b.addActionListener(e -> {
         try {
            img.setText("Loading...");
            String u = dogUrl();
            img.setIcon(new ImageIcon(new URL(u)));
            img.setText(null);
         } catch (Exception ex) {
            img.setText("Failed to load image");
         }
      });
   }
}
```

Step 5 — Compile

cd C:\Practical5\src

javac -d ..\classes MiniClient.java calculator*.java

```
C:\Practical5\src>javac -d ..\classes MiniClient.java calculator\*.java
```

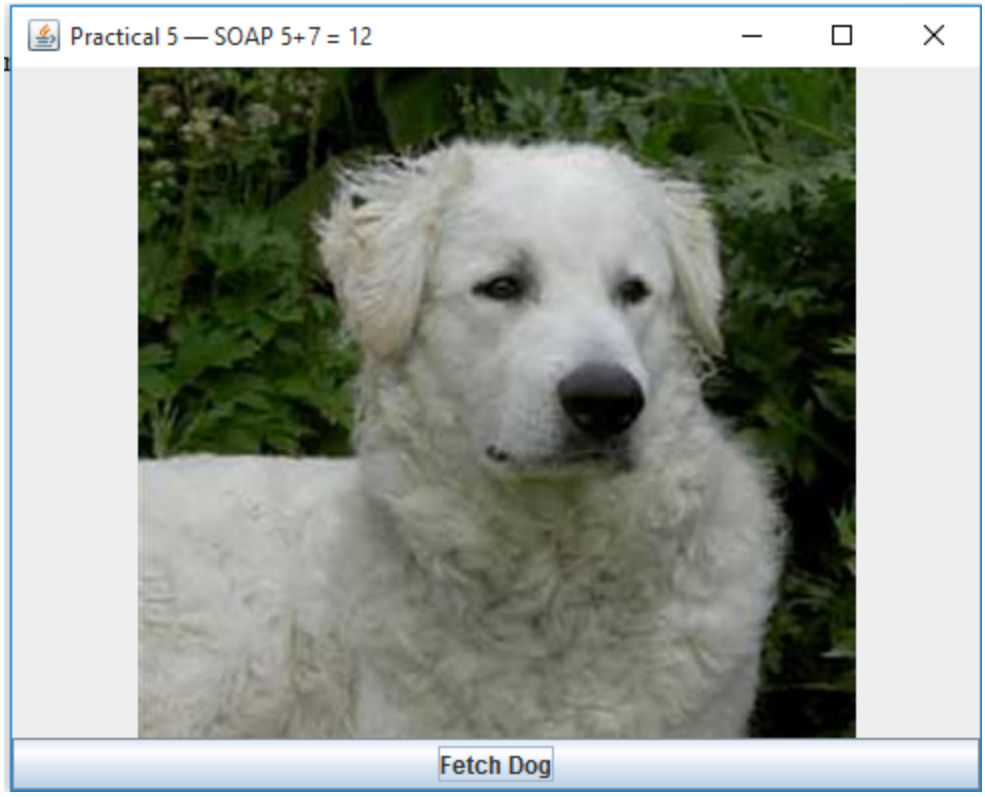
Step 6 — Run

cd ..\classes
java MiniClient

```
C:\Practical5\src>cd ..\classes

C:\Practical5\classes>java MiniClient
```

Output

- Window title: Practical 4 — SOAP 5+7 = 12
- UI with a button: Fetch Dog
- On click → Random dog image appears.



Name: Mansi Ramchandra Mourya
Roll no.: 31011123031
TYCS

Web Services Practical 5

Title

Develop a Servlet that Consumes a Web Service (HTTP Cat API)

Step 1 — Create Project Structure

C:\Practical5
C:\Practical5\WEB-INF
C:\Practical5\WEB-INF\classes
C:\Practical5\WEB-INF\lib

Step 2 — Create Servlet Code

File: C:\Practical5\WEB-INF\classes\HttpCatServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HttpCatServlet extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp)
         throws ServletException, IOException {
      resp.setContentType("text/html;charset=UTF-8");
      PrintWriter out = resp.getWriter();

      String code = req.getParameter("code");
      if (code == null) code = "404";

      String imgUrl = "https://http.cat/" + code;

      out.println("<html><body style='text-align:center;'>");
      out.println("<h2>HTTP Cat Demo — Code " + code + "</h2>");
      out.println("<img src='" + imgUrl + "' alt='Cat for " + code + "' width='500'><br>");
      out.println("<p>Try changing the code in the URL: "
            + "<a href='?code=200'>200</a>, "
            + "<a href='?code=503'>503</a>, "
            + "<a href='?code=418'>418</a></p>");
      out.println("</body></html>");
```

```
  }
}
```

Step 3 — Create web.xml

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
 <servlet>
  <servlet-name>HttpCatServlet</servlet-name>
  <servlet-class>HttpCatServlet</servlet-class>
 </servlet>
 <servlet-mapping>
  <servlet-name>HttpCatServlet</servlet-name>
  <url-pattern>/cat</url-pattern>
 </servlet-mapping>
</web-app>
```

Step 4 — Compile Servlet

```
cd C:\Practical5\WEB-INF\classes
javac -cp ".;C:\apache-tomcat\lib\*" HttpCatServlet.java
```

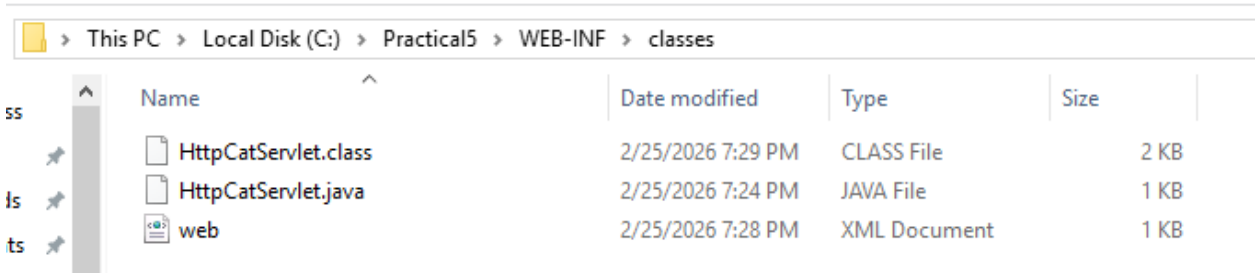
Step 5 — Deploy

Copy the C:\Practical5 folder into C:\apache-tomcat\webapps
Start Tomcat: C:\apache-tomcat\bin\startup.bat

Step 6 — Run in Browser

http://localhost:8080/Practical5/cat?code=200
http://localhost:8080/Practical5/cat?code=404
http://localhost:8080/Practical5/cat?code=503

Output:



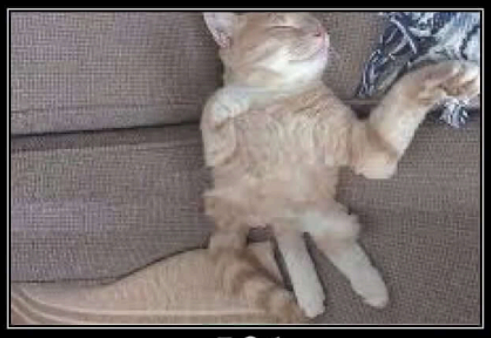
Tomcat

```
25-Feb-2026 19:36:01.541 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying w
eb application directory [C:\apache-tomcat\webapps\ROOT]
25-Feb-2026 19:36:01.557 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment
of web application directory [C:\apache-tomcat\webapps\ROOT] has finished in [16] ms
25-Feb-2026 19:36:01.589 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8080"]

25-Feb-2026 19:36:01.605 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in 1236 ms
```

localhost:8080/Practical5/cat?code=501S

Google Chrome isn't your default browser Set as default

HTTP Cat Demo — Code 501



501

Not Implemented

Try changing the code in the URL: 200, 503, 418

HTTP Cat Demo — Code 503

Try changing the code in the URL: 200, 503, 418

HTTP Cat Demo — Code 200

Try changing the code in the URL: 200, 503, 418

Name:Mansi Ramchandra Mourya

Roll no.: 31011123031

Web Services

Practical No. 6

Title

Define a Web Service Method that Returns Database Contents in JSON (using JSONPlaceholder)

Objective

To develop a Java Servlet web service that retrieves user records from an online API and returns them as JSON, and display them in a tabular format using JavaScript fetch().

Learning Outcomes

After completing this practical, you will be able to: 1. Create a Servlet that consumes a REST API 2. Return JSON from a servlet as a web service response 3. Use JavaScript fetch() to call a servlet and process JSON 4. Display JSON data in HTML table format 5. Understand client–server architecture in web applications

Required Software

- Windows 10 or later
- JDK 8+
- Apache Tomcat 8.x or 9.x
- Internet Connection

Correct Project Structure (VERY IMPORTANT)

Place your project inside Tomcat webapps folder:

```
apache-tomcat/
└── webapps/
    └── Practical6/
        ├── index.html
        └── WEB-INF/
            ├── web.xml
            ├── classes/
            │   └── UserService.class
            └── lib/
```

File Placement Rules

File	Location	Purpose
HTML page	Practical7 folder	Client side interface
web.xml	WEB-INF	Servlet mapping
.class file	WEB-INF/classes	Servlet execution
JAR libraries	WEB-INF/lib	External libraries

NOTE: Files inside WEB-INF cannot be accessed directly by browser.

Servlet Code (UserService.java)

```java
import java.io.*;
import java.net.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class UserService extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        resp.setContentType("application/json;charset=UTF-8");
        PrintWriter out = resp.getWriter();

        try {
            URL url = new URL("https://jsonplaceholder.typicode.com/users");
            HttpURLConnection con = (HttpURLConnection) url.openConnection();
            con.setRequestMethod("GET");

            BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));
            StringBuilder sb = new StringBuilder();
            String line;
            while ((line = in.readLine()) != null) sb.append(line);
            in.close();

            out.println(sb.toString());
        } catch (Exception e) {
            out.println("{\"error\":\"" + e.getMessage() + "\"}");
        }
    }
}
```

---

web.xml Configuration

```xml
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
  <servlet>
    <servlet-name>UserService</servlet-name>
    <servlet-class>UserService</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>UserService</servlet-name>
    <url-pattern>/users</url-pattern>
  </servlet-mapping>
</web-app>
```

Compile the Servlet

Open Command Prompt inside:

```
Practical6\WEB-INF\classes
```

Run:

```
javac -cp ".;C:\apache-tomcat\lib\*" UserService.java
```

Deploy Project

Copy the folder to:

```
C:\apache-tomcat\webapps
```

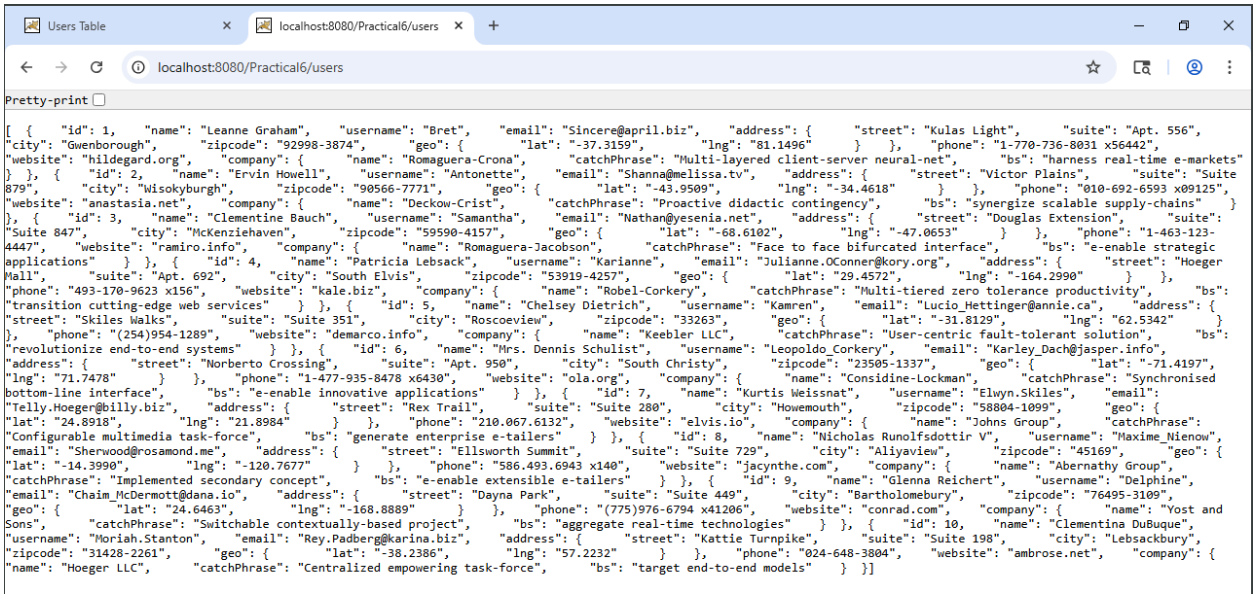
Start Tomcat:

```
C:\apache-tomcat\bin\startup.bat
```

Test the Web Service

Direct Servlet Test

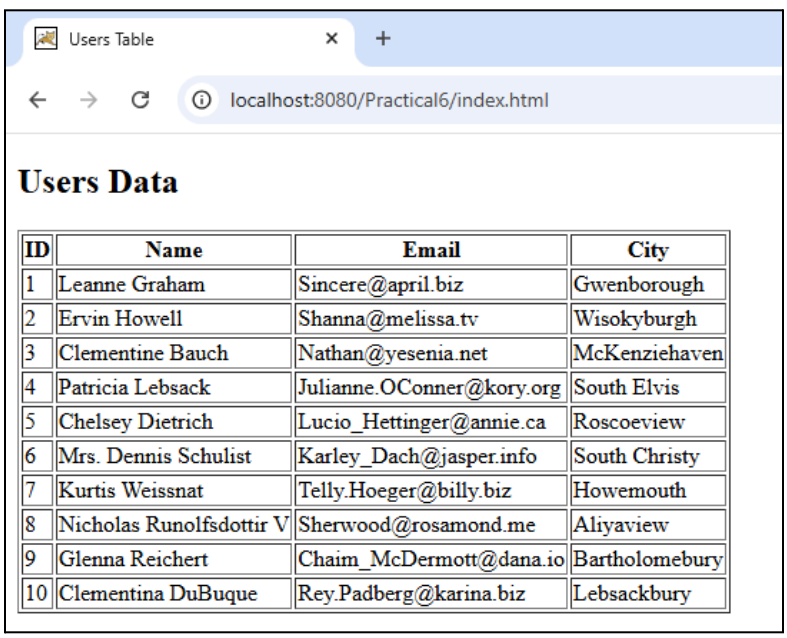
```
http://localhost:8080/Practical6/users
```




```
[ {    "id": 1,    "name": "Leanne Graham",    "username": "Bret",    "email": "Sincere@april.biz",    "address": {      "street": "Kulas Light",      "suite": "Apt. 556",
"city": "Gwenborough",     "zipcode": "92998-3874",      "geo": {        "lat": "-37.3159",        "lng": "81.1496"      }    },    "phone": "1-770-736-8031 x56442",
"website": "hildegard.org",    "company": {      "name": "Romaguera-Crona",      "catchPhrase": "Multi-layered client-server neural-net",      "bs": "harness real-time e-markets"
}  },  {    "id": 2,    "name": "Ervin Howell",    "username": "Antonette",    "email": "Shanna@melissa.tv",    "address": {      "street": "Victor Plains",      "suite": "Suite
879",      "city": "Wisokyburgh",      "zipcode": "90566-7771",      "geo": {        "lat": "-43.9509",        "lng": "-34.4618"      }    },    "phone": "010-692-6593 x09125",
"website": "anastasia.net",    "company": {      "name": "Deckow-Crist",      "catchPhrase": "Proactive didactic contingency",      "bs": "synergize scalable supply-chains"    }
},  {    "id": 3,    "name": "Clementine Bauch",    "username": "Samantha",    "email": "Nathan@yesenia.net",    "address": {      "street": "Douglas Extension",      "suite":
"Suite 847",      "city": "McKenziehaven",      "zipcode": "59590-4157",      "geo": {        "lat": "-68.6102",        "lng": "-47.0653"      }    },    "phone": "1-463-123-
4447",    "website": "ramiro.info",    "company": {      "name": "Romaguera-Jacobson",      "catchPhrase": "Face to face bifurcated interface",      "bs": "e-enable strategic
applications"    }  },  {    "id": 4,    "name": "Patricia Lebsack",    "username": "Karianne",    "email": "Julianne.OConner@kory.org",    "address": {      "street": "Hoeger
Mall",      "suite": "Apt. 692",      "city": "South Elvis",      "zipcode": "53919-4257",      "geo": {        "lat": "29.4572",        "lng": "-164.2990"      }    },
"phone": "493-170-9623 x156",    "website": "kale.biz",    "company": {      "name": "Robel-Corkery",      "catchPhrase": "Multi-tiered zero tolerance productivity",      "bs":
"transition cutting-edge web services"    }  },  {    "id": 5,    "name": "Chelsey Dietrich",    "username": "Kamren",    "email": "Lucio_Hettinger@annie.ca",    "address": {
"street": "Skiles Walks",      "suite": "Suite 351",      "city": "Roscoeview",      "zipcode": "33263",      "geo": {        "lat": "-31.8129",        "lng": "62.5342"      }
},    "phone": "(254)954-1289",    "website": "demarco.info",    "company": {      "name": "Keebler LLC",      "catchPhrase": "User-centric fault-tolerant solution",      "bs":
"revolutionize end-to-end systems"    }  },  {    "id": 6,    "name": "Mrs. Dennis Schulist",    "username": "Leopoldo_Corkery",    "email": "Karley_Dach@jasper.info",
"address": {      "street": "Norberto Crossing",      "suite": "Apt. 950",      "city": "South Christy",      "zipcode": "23505-1337",      "geo": {        "lat": "-71.4197",
"lng": "71.7478"      }    },    "phone": "1-477-935-8478 x6430",    "website": "ola.org",    "company": {      "name": "Considine-Lockman",      "catchPhrase": "Synchronised
bottom-line interface",      "bs": "e-enable innovative applications"    }  },  {    "id": 7,    "name": "Kurtis Weissnat",    "username": "Elwyn.Skiles",    "email":
"Telly.Hoeger@billy.biz",    "address": {      "street": "Rex Trail",      "suite": "Suite 280",      "city": "Howemouth",      "zipcode": "58804-1099",      "geo": {
"lat": "24.8918",        "lng": "21.8984"      }    },    "phone": "210.067.6132",    "website": "elvis.io",    "company": {      "name": "Johns Group",      "catchPhrase":
"Configurable multimedia task-force",      "bs": "generate enterprise e-tailers"    }  },  {    "id": 8,    "name": "Nicholas Runolfsdottir V",    "username": "Maxime_Nienow",
"email": "Sherwood@rosamond.me",    "address": {      "street": "Ellsworth Summit",      "suite": "Suite 729",      "city": "Aliyaview",      "zipcode": "45169",      "geo": {
"lat": "-14.3990",        "lng": "-120.7677"      }    },    "phone": "586.493.6943 x140",    "website": "jacynthe.com",    "company": {      "name": "Abernathy Group",
"catchPhrase": "Implemented secondary concept",      "bs": "e-enable extensible e-tailers"    }  },  {    "id": 9,    "name": "Glenna Reichert",    "username": "Delphine",
"email": "Chaim_McDermott@dana.io",    "address": {      "street": "Dayna Park",      "suite": "Suite 449",      "city": "Bartholomebury",      "zipcode": "76495-3109",
"geo": {        "lat": "24.6463",        "lng": "-168.8889"      }    },    "phone": "(775)976-6794 x41206",    "website": "conrad.com",    "company": {      "name": "Yost and
Sons",      "catchPhrase": "Switchable contextually-based project",      "bs": "aggregate real-time technologies"    }  },  {    "id": 10,    "name": "Clementina DuBuque",
"username": "Moriah.Stanton",    "email": "Rey.Padberg@karina.biz",    "address": {      "street": "Kattie Turnpike",      "suite": "Suite 198",      "city": "Lebsackbury",
"zipcode": "31428-2261",      "geo": {        "lat": "-38.2386",        "lng": "57.2232"      }    },    "phone": "024-648-3804",    "website": "ambrose.net",    "company": {
"name": "Hoeger LLC",      "catchPhrase": "Centralized empowering task-force",      "bs": "target end-to-end models"    }  }]
```


HTML Client Test

```
http://localhost:8080/Practical6/index.html
```



ID	Name	Email	City
1	Leanne Graham	Sincere@april.biz	Gwenborough
2	Ervin Howell	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury

Client HTML Page

```
<!DOCTYPE html>
<html>
<head>
  <title>Users Table</title>
  <script>
    async function loadUsers() {
      let res = await fetch("users");
      let data = await res.json();
      let table = "<table border='1'><tr><th>ID</th><th>Name</th><th>Email</
th><th>City</th></tr>";
      data.forEach(u => {
        table += `<tr>
          <td>${u.id}</td>
          <td>${u.name}</td>
          <td>${u.email}</td>
          <td>${u.address.city}</td>
        </tr>`;
      });
      table += "</table>";
      document.getElementById("output").innerHTML = table;
    }
  </script>
</head>
<body onload="loadUsers()">
  <h2>Users Data</h2>
  <div id="output">Loading...</div>
</body>
</html>
```

Common Student Mistakes & Errors

Mistake	Error Seen	Reason
.class outside classes folder	404 error	Tomcat cannot find servlet
web.xml outside WEB-INF	/users not working	Mapping not loaded
Opening HTML by double click	Table not loading	Server not used
Wrong fetch path	Failed request	Wrong URL
Internet off	JSON error message	API unreachable

Wrong tomcat path in javac	package javax.servlet does not exist	Classpath missing

---

Debugging Checklist (Follow in Order)

1. Tomcat started?
2. Folder inside webapps?
3. /users URL working?
4. .class inside classes folder?
5. Correct fetch path?
6. Internet ON?

Viva Questions

Q: Why WEB-INF folder used?
Security — browser cannot directly access servlet files

Q: Why servlet returns JSON?
Acts as web service API

Q: Why fetch() used?
To call server asynchronously without reloading page

Q: What is JSONPlaceholder?
Fake REST API for testing applications

Q: Difference between database and API?
Database stores data, API provides access to data

Conclusion

You created a servlet-based web service that fetches remote data and displays it dynamically in a browser, demonstrating full client-server interaction.